

Questions

Questions::

=====

1. Application Overview

=====

- Tell me about your application

Our application is an internal auditing app used by companies to perform audit workflow in the proper manner.

Audit is basically maintaining internal information about an entity

For ex your finance department auditing will include fields like risks ,control ,documents

- Who uses it?

Senior/junior auditors ,directors ,managers

- Why do they use it?

To have an insight of what going on inside the company

=====

2. Architecture & Design

=====

- Architecture of the application

Our application follows a hybrid architecture and is currently in a transition phase from a monolithic system to a microservices-based architecture.

At present, we have a single monolithic application that acts as the primary API Gateway. It serves as the main entry point for client requests and handles request routing, authentication, and some shared business logic.

Alongside the monolith, we have identified and segregated certain business functionalities into independent microservices. These microservices are gradually being extracted from the monolith and communicate with it through REST APIs and asynchronous messaging where required.

On the frontend, we use React with TypeScript for building the user interface, Redux for state management, and Vite as the build tool to ensure fast development and optimized builds.

For infrastructure and supporting components:

- Redis is used for caching and performance optimization.
- Kafka is used for asynchronous event-driven communication.
- Docker is used for containerizing application components.
- Kubernetes is used for container orchestration and auto-scaling.
- AWS EC2 is used for hosting application services.
- AWS Lambda is used for specific event-driven or background tasks.

- AWS S3 is used for secure document and file storage.
- Splunk is used for centralized logging.
- Datadog is used for monitoring, metrics, and alerting.

Currently, the application uses a single shared database, which is typical during the initial phase of migrating from a monolith to microservices. This setup simplifies data consistency and reduces operational complexity while the migration is ongoing.

Overall, this architecture enables us to maintain system stability, support scalability, and modernize the application incrementally without impacting existing users.

- Why this architecture?

We chose this hybrid architecture because the application was originally built as a monolith and was already serving active users. Moving directly to a full microservices architecture would have been risky, time-consuming, and could have impacted production stability.

By keeping the monolith as the central entry point and gradually extracting selected modules into microservices, we can modernize the system incrementally without disrupting existing functionality.

This approach allows us to:

- Avoid a big-bang rewrite
- Reduce production risk
- Continue feature development during migration
- Scale only high-traffic or critical modules as microservices
- Validate microservices in production step by step

n6-

Using a single shared database during the transition simplifies data consistency and reduces operational complexity

- Advantages of the architecture

1. Gradual Migration

- Enables step-by-step transition from monolith to microservices
- Avoids risky big-bang rewrites

2. System Stability

- Existing monolith continues to serve users
- No disruption to critical business workflows

3. Independent Scalability

- High-traffic or critical modules can be scaled independently
- Kubernetes supports auto-scaling where required

4. Faster Time to Market

- New features can be developed alongside migration
- Teams are not blocked by architectural changes

5. Reduced Operational Risk

- Issues in new microservices do not impact the entire system
- Easier rollback during failures

6. Better Resource Utilization

- Microservices consume resources only when needed
- Redis and Kafka reduce unnecessary load on core services

7. Improved Observability

- Centralized logging with Splunk
- Real-time monitoring and alerts via Datadog

8. Cloud Readiness

- Docker and Kubernetes make the system cloud-native
- AWS services enable elasticity and scalability

9. Performance Optimization

- Caching with Redis reduces database load
- Event-driven processing via Kafka improves responsiveness

10. Future Scalability

- Architecture is ready for full microservices adoption
- Database separation and service isolation can be added incrementally

- Disadvantages of the architecture

1. Increased Complexity

- Managing both monolith and microservices increases architectural complexity
- Requires careful coordination between components

2. Single Database Limitation

- Shared database creates tight coupling between services
- Limits independent schema evolution
- Becomes a scalability bottleneck

3. Operational Overhead

- Requires DevOps effort for Docker, Kubernetes, monitoring, and logging
- More moving parts to manage and maintain

4. Network Latency

- Inter-service communication introduces network calls
- Slower compared to in-process monolith calls

5. Data Consistency Challenges

- Harder to manage transactions across services
- Distributed transaction handling is complex

6. Debugging Difficulty

- Issues can span multiple services
- Requires strong logging, tracing, and correlation IDs

7. Partial Microservices Benefits

- Cannot fully leverage microservices advantages due to shared database
- True service isolation is not achieved yet

8. Learning Curve

- Teams need expertise in cloud, containers, and distributed systems
- Higher onboarding time for new developers

9. Cost Overhead

- Kubernetes, monitoring tools, and cloud resources increase infrastructure cost

10. Dependency Management

- Monolith remains a critical dependency
- Failure in the monolith can still impact the entire system

=====

3. API Gateway & Traffic Management

=====

- Have you used an API Gateway?

Yes, we use an API Gateway in our application.

Currently, the monolithic application itself acts as the API Gateway. It serves as the single entry point for all client requests and is responsible for request routing, authentication, authorization, and forwarding requests to the appropriate internal microservices.

This approach fits our current transition phase from monolith to microservices. It allows us to centralize cross-cutting concerns like security, logging, and validation, while gradually extracting business functionalities into independent microservices.

As the system evolves, the plan is to move toward a dedicated API Gateway solution, but for now the monolith effectively fulfills the API Gateway role.

- How does the API Gateway manage load?

The API Gateway manages load by acting as a centralized traffic controller for all incoming client requests.

It distributes requests across multiple instances of backend services, ensuring that no single service instance becomes a bottleneck. When deployed with Kubernetes, the gateway works with the platform's built-in load balancing to route traffic to healthy pods only.

The gateway also helps manage load by:

- Enforcing rate limiting to prevent request spikes
- Applying throttling to slow down excessive traffic
- Validating requests early to avoid unnecessary backend processing
- Rejecting invalid or unauthorized requests at the entry point

Additionally, auto-scaling mechanisms increase or decrease service instances based on CPU, memory, or request volume, allowing the system to handle varying traffic levels efficiently.

Auto-scaling:

Auto-scaling allows the application to automatically increase or decrease the number of service instances based on load. Using Kubernetes, services scale horizontally based on metrics like CPU usage, memory consumption, or request count. This ensures the application can handle traffic spikes while optimizing resource usage during low traffic periods.

Throttling:

Throttling controls the rate at which requests are processed when traffic exceeds a defined threshold. Instead of completely rejecting requests, the system slows down excessive traffic to protect backend services from overload and maintain overall system stability.

Rate Limiting:

Rate limiting restricts the number of requests a client can make within a specific time window. It helps prevent abuse, ensures fair usage among clients, and protects the system from excessive or malicious traffic.

Throttling and rate limiting are handled at the API Gateway level.

4. Database Design & Load Handling

- How many databases do you have?

Currently, we use a single shared database across the application.

This is intentional because the system is in a transition phase from a monolithic architecture to microservices. Using a single database simplifies data consistency, reduces migration complexity, and ensures stability while services are being gradually extracted.

As part of the future roadmap, the plan is to move toward a database-per-service model once the microservices are fully isolated.

- How does your database handle load?

The database handles load through a combination of design optimizations and supporting infrastructure.

We use proper indexing on frequently queried columns to speed up read operations. Connection pooling is enabled to efficiently manage database connections and prevent connection exhaustion under high traffic.

To reduce direct database load, Redis caching is used for frequently accessed and read-heavy data. Pagination is applied for large result sets to avoid heavy queries.

Additionally, background processing and asynchronous workflows via Kafka help offload non-critical operations from synchronous database access, improving overall performance and scalability.

Connection Pooling

Connection pooling works by maintaining a pool of pre-created database connections that can be reused by multiple requests instead of creating a new connection each time.

When the application starts, the connection pool (for example, HikariCP in Spring Boot) creates a fixed number of database connections and keeps them ready.

When a request needs to access the database:

- It borrows an available connection from the pool
- Executes the query
- Returns the connection back to the pool instead of closing it

If all connections are in use, new requests wait until a connection becomes available, rather than creating unlimited connections and overwhelming the database.

This approach:

- Reduces connection creation overhead
- Improves performance under high traffic

- Prevents database connection exhaustion
- Ensures controlled and efficient use of database resources

=====

5. Caching Strategy

=====

- Is caching used in your project?

Yes, caching is used in our project.

We use Redis as a centralized in-memory cache to improve performance and reduce load on the database. Caching is mainly applied to frequently accessed, read-heavy data that does not change frequently.

- What is stored in the cache?
- When is data stored in cache?
- When and how is cache cleared?

=====

6. Logging & Monitoring

=====

- How is logging implemented?
- Which logging/monitoring tools are used?
- How is the logging tool integrated?
- How does application data reach the logging system?

=====

7. Authentication & Authorization

=====

- How is authentication managed?
- How is authorization managed?

=====

8. Inter-Service Communication

=====

- How do services communicate with each other?
- Synchronous vs asynchronous communication
- where is multithreading applied in your application ?

=====

9. Messaging / Event Streaming

=====

- Where is Kafka used in your project?
- Why is Kafka used?

=====

10. Background & Scheduled Jobs

=====

- Are there background jobs in your project?
- What kind of tasks run in the background?

=====

11. Bugs & Issue Handling

=====

- Bugs faced in the project
- How were those bugs identified?
- How were they resolved?

=====

12. Recent Contributions

=====

- Your recent contributions to the project

=====

13. Roles & Responsibilities

=====

- Your role in the team
- Your day-to-day responsibilities

=====

14. Challenges & Problem Solving

=====

- Difficulties faced in the project
- How did you tackle them?

=====

15. AOP (Aspect-Oriented Programming)

=====

- Where is AOP used in your project?
- Why was AOP used?

=====

Cloud

=====

- how do you manage load balancing ?

=====
Others
=====

1) HTTP vs HTTPS

Functional Info

We have an **internal audit application** that supports the **end-to-end audit lifecycle**.

Key modules:

- Dashboard
- Assessment area
- Project management
- Time tracking
- Audit report creation
- Recycle bin
- Audit history

Core audit objects:

- Risks and controls
- Strategic risks
- Objectives
- Workpapers

Major functionalities:

- Configurable **workflows**
- **Dynamic role- and permission-based access control**
- **Dynamic rule engine** for business logic
- **Workpaper upload and download**
- Customizable **naming and taxonomy**
- **SMTP-based email notifications**
- Automated **audit report generation**

Business user/admin flow

Admin Flow (Context Definition)

In the audit application, the **Admin** has full system-level access and configuration control.

Admin responsibilities include:

- Managing **taxonomy settings** across the application
- Defining **roles and permissions**
 - Creating new roles from existing system roles
 - Assigning permissions to roles
- **User management**
 - Creating users
 - Assigning roles to users
- Customizing the system
 - Renaming audit objects (risk, control, objective, workpaper, etc.)
 - Modifying taxonomy structures
- Configuring **SMTP settings** for email notifications

In addition to administrative capabilities, the **Admin can perform all operations available to a normal user**, including working on audits, assessments, workpapers, workflows, and reports.

User Flow (Context Definition)

A **User** has functional access based on the roles and permissions assigned to them.

User capabilities include:

- Accessing the **Assignment area**
 - Creating new assignments
 - Adding audit objects within an assignment
- Working with **Projects**
 - Linking a project to a specific audit area
 - Performing audits within that project
 - Adding predefined audit objects (risks, controls, objectives, workpapers, etc.)
- Managing audit objects
 - Creating, editing, and deleting objects
- Generating **audit reports** from assignment or project data
- Participating in **assessment workflows**
 - Being assigned to specific assessment phases
 - Receiving notifications based on assigned phases and tasks

User limitations:

- No access to **administrative areas** such as:
 - System settings
 - Statistics
 - Configuration modules

Application flow

TeamMate+ Audit – Overview

TeamMate+ Audit is a software application used by internal audit teams to **plan, execute, track, and report audits** efficiently.

It is widely used by organizations—especially in regulated industries like **banking**—to manage **risk, compliance, and governance** activities.

Who Uses TeamMate+ Audit?

1. Internal Audit Teams

- Conduct audits across departments
- Evaluate internal controls and processes
- Identify gaps, risks, and improvement areas

2. Risk Management Professionals

- Identify and assess operational, financial, and compliance risks
- Ensure risks are monitored and mitigated

3. Compliance Officers

- Ensure adherence to regulatory requirements and internal policies
- Track compliance issues and corrective actions

4. Audit Managers & Supervisors

- Plan audits and allocate resources
 - Monitor audit progress and quality
 - Review findings and final reports
-

Why Do Organizations Use TeamMate+ Audit?

Streamlined Audit Processes

- End-to-end audit lifecycle management
- Reduces manual work and spreadsheets

Enhanced Collaboration

- Centralized platform for auditors and stakeholders
- Clear visibility into tasks, findings, and status

Risk-Based Auditing

- Focuses audits on high-risk areas
- Improves early risk detection

Compliance Assurance

- Ensures audits align with regulatory standards
 - Maintains audit trails and documentation
-

Banking Sector Example (Simple Explanation)

Q1. Who Uses TeamMate+ Audit in a Bank?

- **Internal Audit Team**
Checks whether banking operations are safe, efficient, and compliant.
 - **Risk Management Team**
Identifies risks like fraud, credit risk, system failures, and operational issues.
 - **Compliance Officers**
Ensure adherence to RBI / central bank regulations and internal policies.
-

Q2. Why Do Banks Use TeamMate+ Audit?

Banks handle:

- Huge volumes of transactions
- Sensitive customer data
- Strict regulatory requirements

They must ensure:

- **Operational safety** – no fraud or transaction errors
- **Regulatory compliance** – RBI / central bank rules
- **Risk minimization** – early detection of issues
- **Accurate audit reporting** – for regulators and senior management

TeamMate+ Audit helps **automate, standardize, and control** these activities.

Q3. How Does a Bank Use TeamMate+ Audit?

1. Audit Planning

- Decide audit areas (loans, deposits, treasury, IT systems)
- Schedule audits and assign auditors
- Prioritize audits based on risk levels

2. Risk Assessment

- Identify high-risk areas:
 - Credit risk
 - Fraud risk
 - Operational risk
- Allocate more audit effort to critical areas

3. Audit Execution

- Auditors review transactions and controls
- Evidence and findings stored centrally
- Eliminates paperwork and scattered files

4. Reporting

- Automatically generates audit reports
- Clearly highlights:
 - Issues
 - Risks
 - Recommendations

5. Follow-Up (Cron-Based Tracking)

- Tracks whether audit issues are resolved
- Sends reminders until actions are closed
- Ensures accountability

Benefits to the Bank

- **Time Savings** – Less manual tracking and documentation
 - **Reduced Risk** – Systematic and consistent audits
 - **Improved Compliance** – Easier regulatory adherence
 - **Actionable Insights** – Identifies recurring issues and trends
 - **Transparency** – Clear visibility for management and regulators
-

Final Summary

✓ In short:

In the banking sector, **TeamMate+ Audit** is used to **manage audits, control risks, ensure regulatory compliance, and improve audit efficiency**, helping banks remain **safe, compliant, and trustworthy**.

Technical Architecture

Tech stack used

Frontend:

- React 18, TypeScript, Vite
- Redux Toolkit

Backend:

- Java 17
- Spring Boot 3.x
- Spring Security, JWT, OAuth2
- RESTful APIs, Swagger (OpenAPI)
- JPA / Hibernate

Database:

- MySQL

Messaging & Cache:

- Kafka
- Redis

DevOps & Cloud:

- Git, GitHub
- Jenkins (CI/CD pipelines)
- Docker, Kubernetes
- AWS (EC2, S3, Lambda)

Testing & Quality:

- JUnit, Mockito
- Spring Boot Test
- SonarQube

Monitoring & Logging:

- Splunk
- Datadog

Tools:

- MobaXterm
- PuTTY

Design Pattern

Adapter design pattern::

Adapted Design Pattern – Application Flow Explanation

Our application follows an **adapted Adapter-based design pattern** to bridge HTTP requests with the service layer and enforce a structured business pipeline.

Why we use this pattern

- To **decouple controllers from business logic**
 - To enforce a **standard execution pipeline** for every request
 - To centralize **authorization, DB handling, and post-processing**
 - To make the system **extensible and maintainable**
-

Request Flow (End-to-End)

1. Controller Layer

- Receives the HTTP request
- Performs minimal request validation
- Creates an instance of the **Adapter**

2. Adapter Layer

- Acts as a bridge between the HTTP layer and business logic
- Translates request data into a business-friendly format
- Passes control to the **Execution Service**
- Ensures the controller remains thin

3. Execution Service (Business Pipeline)

- Executes a standardized pipeline:
 - User authentication & role validation

- Permission checks
- Connection handling
- Prepared statement creation
- Database operations
- Post-operation validations (audit logs, response checks, cleanup)

4. **Specific Business Service**

- Contains actual business logic
- Executes domain-specific rules
- Interacts with the database layer

5. **Database Layer**

- Performs CRUD operations using prepared statements
- Returns results to the service layer

6. **Post-Processing**

- Result validation
- Error handling
- Audit/logging (if applicable)
- Response preparation

7. **Response Flow**

- Travels back through Execution Service → Adapter → Controller
 - Controller sends the final HTTP response
-

High-Level Flow Summary

Controller



Adapter



Execution Service (Business Pipeline)



Specific Service



Database



Post-Operation Checks



Response back to Controller

Key Benefits

- **Loose coupling** between layers
- **Single execution pipeline** for all business flows
- **Easy to add new services** without touching controllers
- **Cleaner controllers, reusable business logic**
- **Consistent security and DB handling**

If you want, I can:

- Convert this into **interview talking points**
- Map it to **Spring Boot components**
- Show **which GoF patterns this adapts (Adapter + Template Method)**
- Rewrite it as **architecture documentation**

Just say the word.

give an example

Alright, here's a **concrete, no-fluff example** using **Spring Boot + Java**, mapped exactly to the flow you described. This is the kind of example interviewers actually understand.

Example: Create Audit Assignment Flow

1 Controller (Thin – no business logic)

```
@RestController
@RequestMapping("/assignments")
public class AssignmentController {

    private final AssignmentAdapter assignmentAdapter;

    public AssignmentController(AssignmentAdapter assignmentAdapter)
    {
        this.assignmentAdapter = assignmentAdapter;
    }

    @PostMapping
    public ResponseEntity<?> createAssignment(@RequestBody
AssignmentRequest request) {
        return assignmentAdapter.createAssignment(request);
    }
}
```

👉 Controller only:

- Accepts HTTP request
- Delegates to Adapter
- No DB, no security, no logic

2 Adapter (HTTP → Business Bridge)

```
@Component
public class AssignmentAdapter {

    private final ExecutionService executionService;
```

```

    public AssignmentAdapter(ExecutionService executionService) {
        this.executionService = executionService;
    }

    public ResponseEntity<?> createAssignment(AssignmentRequest
request) {

        AssignmentContext context = new AssignmentContext();
        context.setRequest(request);

        ExecutionResult result = executionService.execute(
            context,
            BusinessOperation.CREATE_ASSIGNMENT
        );

        return ResponseEntity.status(result.getStatus())
            .body(result.getResponse());
    }
}

```

👉 Adapter responsibilities:

- Converts HTTP request to **business context**
- Calls execution pipeline
- Converts business response → HTTP response

3 Execution Service (Business Pipeline – Core)

This is where your **adapted design pattern** really lives.

```

@Service
public class ExecutionService {

    private final AuthService authService;
    private final BusinessServiceFactory serviceFactory;

    public ExecutionResult execute(BusinessContext context,

```

```

        BusinessOperation operation) {

    // 1. Authentication & Role validation
    authService.validateUser(context.getUser(), operation);

    // 2. Get business service
    BusinessService service =
        serviceFactory.getService(operation);

    // 3. Execute business logic
    Object result = service.execute(context);

    // 4. Post-operation checks
    audit(context, result);

    return ExecutionResult.success(result);
}

private void audit(BusinessContext context, Object result) {
    // logging / audit trail / validation
}
}

```

👉 Pipeline enforced here:

- Security
- Service routing
- Execution
- Post-processing

4 Business Service Interface (Template-like)

```

public interface BusinessService {
    Object execute(BusinessContext context);
}

```

5 Specific Business Service (Actual Logic)

```
@Service
public class CreateAssignmentService implements BusinessService {

    private final AssignmentRepository repository;

    public CreateAssignmentService(AssignmentRepository repository)
    {
        this.repository = repository;
    }

    @Override
    public Object execute(BusinessContext context) {

        AssignmentRequest req =
            ((AssignmentContext) context).getRequest();

        // Prepared statement via JPA / JDBC
        Assignment assignment = new Assignment();
        assignment.setName(req.getName());
        assignment.setCreatedBy(context.getUser().getId());

        return repository.save(assignment);
    }
}
```

👉 Only **business logic**, nothing else.

6 Factory (Routes Operation → Service)

```
@Component
public class BusinessServiceFactory {

    private final Map<BusinessOperation, BusinessService> services;

    public BusinessServiceFactory(List<BusinessService> serviceList)
    {
        services = Map.of(
            BusinessOperation.CREATE_ASSIGNMENT,
            serviceList.stream()
                .filter(s -> s instanceof CreateAssignmentService)
                .findFirst()
                .orElseThrow()
        );
    }
}
```

```

        .filter(s -> s instanceof CreateAssignmentService)
        .findFirst()
        .orElseThrow()
    );
}

public BusinessService getService(BusinessOperation operation) {
    return services.get(operation);
}
}

```

7 Flow Recap (Interview-friendly)

HTTP Request

↓

Controller (thin)

↓

Adapter (HTTP → Business)

↓

Execution Service (pipeline)

↓

Specific Business Service

↓

Database

↑

Post-operation checks

↑

HTTP Response

Patterns Used (Tell it straight)

- **Adapter Pattern** – Controller → Adapter → Service
- **Template Method (adapted)** – Fixed execution pipeline
- **Factory Pattern** – Resolve business service at runtime
- **Single Responsibility** – Each layer does one thing

Roles and responsibility

Roles and responsibility

=====

- Actively involved in development activities, with approximately 80% focus on new feature development and enhancements.
- Handle around 20% of sprint capacity for production support, client-reported defects, and bug fixes, ensuring minimal impact on business operations.
- Participate in technical and architectural discussions, contributing to design decisions, scalability considerations, and best practices.
- Actively participate in Agile Scrum ceremonies including:
 - Daily stand-ups
 - Sprint planning
 - Backlog grooming
 - PI increment planning
- Perform peer code reviews to ensure code quality, adherence to coding standards, performance optimization, and maintainability.

SDLC

Software Development Life Cycle (SDLC)

1. Feature discussion and high-level planning are carried out by higher management and product leadership.
2. Approved features are broken down into user stories (tickets) by the entire team, including developers and QA.
3. Steps 1 and 2 are part of PI (Program Increment) Planning, which is done for the upcoming 3 months of development and QA work.
4. Before the start of every sprint, stories are groomed, refined, estimated, and assigned to respective developers and QA engineers.
5. During sprint planning, stories are assigned to developers. Based on the number of stories (for example, 10 tickets) and available QA resources (for example, 2 QA), QA planning is done for functional and feature-wise testing.
6. Story estimation is done using story points, where:
 - 1 story point represents approximately 1 day (8 hours) of work
 - A story typically contains 2 to 3 story points
7. Once development for a story is completed, it is handed over to the QA team for functional and regression testing.
8. If QA identifies any issues, they are reported as internal bugs.
9. If a bug is related to a story developed by a specific developer, the bug is assigned back to the same developer for fixing.
10. After bug fixes, the QA team re-tests the functionality to ensure the issue is resolved and no regressions are introduced.
11. Once QA sign-off is completed, the feature is tested in the UAT environment by the Product Manager or business stakeholders.
12. As the PI approaches completion, a code freeze is applied on the staging branch.
13. After code freeze: (checked once)
 - Automation testers and developers run automated test suites
 - Developers execute unit and integration test cases
 - DevOps runs integration pipelines
 - Performance team conducts performance and load testing
14. Development activities continue on the main/UAT branch while the staging branch is used for testing and sanity validation.
15. After all validations are successful, the final build is released to production.

SDLC Flow for Client-Reported Bug

1. A bug is reported by the client and logged as a production issue or client defect in the tracking system.
2. The issue is analyzed by the team for impact, severity, and priority.
3. Based on the analysis, a dedicated team or developer is assigned to handle the bug.
4. The bug is planned and included in the next sprint under the allocated support capacity (typically 20% of the sprint).
5. The assigned developer works on the fix, following standard development practices including code changes and unit testing.
6. Once the fix is completed, the bug is moved to QA for validation and regression testing.
7. If QA finds any issues, the bug is reassigned to the developer for correction.
8. After successful QA validation, the fix is deployed to UAT for verification by the product manager or client (if required).
9. Once approved, the fix is merged into the staging branch and included in the next planned release.
10. After release, the fix is monitored in production to ensure the issue is fully resolved and no new issues are introduced.

Feature Deployment

RELEASE MANAGEMENT & DEPLOYMENT FLOW

1. FEATURE DEVELOPMENT PHASE

- Developers work on individual feature branches created from the main development branch.
- Each feature follows:
 - Code implementation
 - Unit testing
 - Local verification
 - Peer code reviews (PRs)
- All merges are done only after review approval and static checks.

2. CODE FREEZE

- Once all planned features for the release are completed, a code freeze is declared.
- During code freeze:
 - No new features are allowed.
 - Only critical bug fixes, security fixes, or release blockers are permitted.
 - Any fix requires explicit approval.
- This ensures stability and prevents scope creep before release.

3. STAGING BRANCH CREATION

- After code freeze, a staging branch is created from the stabilized development branch.
- The staging branch represents a production-like snapshot of the application.
- No direct feature development happens on this branch.

4. STAGING ENVIRONMENT DEPLOYMENT

- The staging branch is deployed to the staging environment.
- This environment closely mirrors production in terms of:
 - Infrastructure
 - Configuration
 - Data structure
 - Security settings

5. AUTOMATION & TEST EXECUTION

- Once deployed to staging, the CI/CD pipeline is triggered.
- The following automated tests are executed:
 - Unit tests
 - Integration tests
 - API contract tests
 - Regression test suites
 - Security and vulnerability scans (if applicable)
- Any test failure blocks the release and sends feedback to the team.

6. MANUAL & BUSINESS VALIDATION

- After automation passes, manual testing is performed:
 - Functional testing
 - Exploratory testing
 - User acceptance testing (UAT)

- Business stakeholders validate:
 - Feature correctness
 - Data integrity
 - End-to-end workflows

7. RELEASE BRANCH CREATION

- Once staging is fully validated, a release branch is created from the staging branch.
- This branch is strictly controlled and used only for:
 - Final bug fixes
 - Versioning
 - Release documentation
- No new features are allowed at this stage.

8. PRODUCTION DEPLOYMENT

- The release branch is deployed to the production environment via the CI/CD pipeline.
- Deployment strategies may include:
 - Blue-green deployment
 - Rolling deployment
 - Canary releases
- Health checks and smoke tests are executed post-deployment.

9. POST-DEPLOYMENT VERIFICATION

- After production deployment:
 - Application logs are monitored
 - Metrics and alerts are observed
 - Performance and error rates are validated
- Immediate rollback plans are kept ready in case of issues.

10. RELEASE CLOSURE

- Once production stability is confirmed:
 - The release branch is tagged
 - Changes are merged back to the main branch
 - Release notes are finalized
- The system is now ready for the next development cycle.

KEY BENEFITS OF THIS PROCESS:

- Controlled and predictable releases
- Early detection of defects
- Production stability
- Clear separation of development, validation, and release stages
- Reduced risk during deployment

Bugs and resolutions

Bugs faced in the project

1) Latency bug

BUG SCENARIO: PERFORMANCE / LATENCY ISSUE – TIME TRACKING MODULE

ISSUE:

Users were experiencing significant latency while loading the Time Tracking screen. The page was attempting to load a large dataset (~5000 records) in a single request, resulting in slow response times and poor user experience.

INITIAL ANALYSIS:

- The first assumption was a database performance issue.
- We analyzed:
 - SQL queries
 - Execution plans
 - Index usage
 - DB response time
- Database queries were optimized and performing within acceptable limits.
- Conclusion: DB was NOT the bottleneck.

DEEPER INVESTIGATION:

- We performed end-to-end profiling of the request flow.
- Identified that the delay was occurring in the application layer.
- Root cause was found in a custom DTO mapper.

ROOT CAUSE:

- A custom mapper was using a traditional for-loop to map each entity to a DTO one-by-one.
- This resulted in:
 - High CPU usage
 - Increased object creation
 - Long processing time for large datasets
- The mapping logic was $O(n)$ and executed for all 5000 records at once, causing noticeable latency.

SOLUTION IMPLEMENTED:

1. DATA INDEXING

- Added proper indexing on the ID columns involved in the time tracking queries.
- Reduced lookup and join cost during data retrieval.

2. PAGINATION IMPLEMENTATION

- Introduced server-side pagination.
- Instead of loading all 5000 records at once, data is now fetched in chunks (e.g., 50 or 100 records per page).
- Significantly reduced payload size and processing time.

3. MAPPER OPTIMIZATION

- Refactored the custom mapper to avoid inefficient looping where possible.
- Optimized DTO mapping to handle smaller paginated datasets efficiently.

RESULT:

- Page load time is reduced drastically.
- CPU and memory utilization normalized.
- Improved overall user experience.
- Application became scalable for larger datasets.

KEY LEARNINGS:

- Performance issues are not always database-related.
- Application-layer processing (DTO mapping, loops, object creation) can become a major bottleneck.
- Pagination and indexing are critical for handling large datasets efficiently.

2)BUG SCENARIO: UI FIELD NOT RENDERING – DATA COMPATIBILITY ISSUE

ISSUE:

Users reported that a specific text field was not visible on the UI while loading a particular screen. The field existed in the layout configuration but was either blank or not rendered properly in the new UI.

IMPACT:

- Users were unable to view or edit a critical test-related field.
- Issue was intermittent and data-specific, making it difficult to reproduce.
- Affected only a subset of existing records.

INITIAL INVESTIGATION:

- Attempted to reproduce the issue locally and in lower environments.
- Verified:
 - UI configuration
 - Field visibility rules
 - Role-based permissions
 - API response payload
- The issue could NOT be reproduced consistently.
- New records were working fine, indicating the problem was not in the UI logic itself.

HYPOTHESIS:

- After detailed discussion and comparison between affected and non-affected records, we suspected a data-related issue rather than a UI or backend logic issue.

DATA ANALYSIS:

- Identified the database column backing the problematic text field.
- Executed queries to extract all distinct values from that column for affected records.
- While analyzing the data, we found:
 - Presence of restricted / non-printable / legacy special characters
 - Characters introduced from an older version of the UI

- These characters were not compatible with the new UI rendering engine

ROOT CAUSE:

- The old UI allowed certain unsupported characters to be persisted in the database.
- The new UI framework had stricter rendering and validation rules.
- As a result, the field failed to render when encountering those invalid characters.

SOLUTION IMPLEMENTED:

1. DATA CLEANUP SCRIPT

- Wrote a backend SQL/data conversion script to sanitize the affected column.
- Removed or replaced unsupported and non-renderable characters.
- Ensured data consistency without impacting valid content.

2. PREVENTIVE VALIDATION

- Added validation at the backend layer to prevent such characters from being saved in the future.
- Ensured backward compatibility with the new UI.

RESULT:

- The affected field started rendering correctly for all users.
- No UI changes were required.
- Issue resolved permanently across environments.

KEY LEARNINGS:

- Legacy data can break modern UI frameworks.
- Not all UI issues are UI bugs — data integrity plays a critical role.
- Proper data validation and cleanup are essential during UI migrations.

3)

- How were those bugs identified?

- How were they resolved?

Authentication

Authentication in our project is implemented using JWT (JSON Web Token) and follows a stateless authentication model.

The application is stateless, and authentication is handled at the API Gateway level. Spring Security is used to validate and authenticate requests using a chain of security filters.

When a user logs in:

- If no JWT is present, the API Gateway validates the user credentials (user ID and password) against the database.
- Once the credentials are successfully verified, the system generates a JWT.

The generated JWT contains:

- User ID
- User roles
- Token metadata (expiry, issuer, etc.)

Two types of tokens are issued:

1. Access Token
2. Refresh Token

The Access Token:

- Is short-lived
- Is stored in the frontend Redux store
- Is sent with every API request in the Authorization header
- Is used to authenticate and authorize the user for each request

The Refresh Token:

- Is long-lived
- Is stored securely in browser cookies
- Is used only when the access token expires or becomes invalid

For every incoming request:

- The API Gateway validates the access token using Spring Security filters
- The roles embedded in the access token are checked against the requested API and request context
- If the token is missing, expired, or role validation fails, the request is rejected

If the access token is expired:

- The refresh token is sent to generate a new access token
- If the refresh token is also invalid, the user is logged out and access is denied

If any token validation fails or a mismatch is detected between the token roles and the requested resource, the JWT is invalidated and the user is blocked from accessing the application with JWT.

Overall, authentication is enforced through a combination of JWT-based stateless security and Spring Security filter chains.

Authorization

Authorization in our project is implemented using a role-to-permission based access control model.

Each role in the system is mapped to a defined set of permissions for specific functional areas of the application. These permissions control access at both API and business logic levels.

When a user is authenticated:

- The user's roles are extracted from the JWT
- Role-to-permission mappings are fetched from the database
- These mappings are cached in Redis to avoid repeated database calls

During request processing:

- Before entering any controller or service layer, authorization checks are enforced
- Access is validated by matching the user's roles against the required permissions for the requested resource or operation

To enforce authorization consistently across the application, we use a custom AOP (Aspect-Oriented Programming) solution:

- Custom annotations are defined on controllers or service methods
- An authorization aspect intercepts method execution
- The aspect retrieves user roles and permissions from the Security Context
- Permissions are validated before allowing method execution

If the required permission is not present:

- The request is blocked
- An authorization exception is thrown
- Access is denied without executing business logic

For performance and consistency:

- Role to permission data is cached in Redis
- Cache is refreshed or invalidated when role or permission changes occur
- Redis ensures fast lookup for every request and subsequent authorization checks

The authenticated user's role and permission details are also stored in the Spring Security Context. This allows:

- Reuse of authorization data across the application
- Easy access during service calls, AOP checks, and logging
- Consistent enforcement of security rules throughout the request lifecycle

Overall, this approach ensures centralized, secure, and scalable authorization with minimal performance overhead while maintaining strict access control.

What is security context ::

SecurityContext is a core Spring Security component that holds the security-related information of the currently authenticated user for the duration of a request.

It contains details such as:

- Authenticated user identity (user ID / username)
- User roles and authorities (permissions)
- Authentication status
- Security credentials (not passwords, but authentication metadata)

In our project, once a JWT is successfully validated:

- User details and roles are extracted from the token
- An Authentication object is created
- This object is stored inside the SecurityContext

The SecurityContext is then accessible throughout the application during that request, including:

- Controllers
- Services
- AOP aspects
- Authorization checks

Spring Security manages the lifecycle of the SecurityContext automatically:

- It is created at the beginning of the request
- Populated after successful authentication
- Cleared after the request is completed

In simple terms:

SecurityContext is the place where Spring Security stores "who the user is and what they are allowed to do" for the current request.